

⚡ Peer-to-Peer Energy Trading Platform — Hackathon Plan

Theme: Crypto & Blockchain

Duration: 52 Hours

Stack: Next.js (fullstack) + Solidity/Foundry + ESP32 IoT

Blockchain: Ethereum Sepolia Testnet

1. How It Works (Simple Explanation for Judges)

🏠 The Problem

Imagine **Ravi** has solar panels on his roof. On a sunny afternoon, his panels produce **more electricity than he needs**. Right now, that extra energy either goes to waste or gets sold back to the utility at a very low price. Meanwhile, consumers like **Priya** are paying full price to the electricity company.

What if Ravi could feed his surplus energy into a grid — and consumers like Priya could draw from it and pay Ravi directly, with no middleman?

⚡ Our Solution

We built a **decentralized energy grid** where:

1. **Ravi (the Producer)** plugs in a smart meter (ESP32 device) to his solar setup. The meter automatically reads how much energy he's producing and consuming — and sends this data to our platform every few seconds.
2. Ravi opens our website, **connects his crypto wallet** (like MetaMask), and registers as the **Producer**.
3. The platform shows Ravi his **live production data** from the smart meter. He can see he has surplus energy. He clicks **"Feed Grid"** and his surplus electricity flows into the grid.
4. **Priya** (and other consumers) connect their wallets, register as **Consumers**, and open the **Grid Dashboard**. They can see the available energy in the grid — fed by Ravi's surplus. The price adjusts dynamically based on supply and demand — this is our **dynamic pricing**.
5. Priya clicks **"Buy 20 kWh from Grid"**. A **smart contract on the Ethereum Sepolia blockchain** automatically:
 - Checks that the grid has 20 kWh available (from Ravi's surplus)
 - Calculates the total price
 - Transfers the crypto payment from Priya to Ravi (the producer)
 - Records the entire trade permanently on the blockchain
6. **Ravi gets paid automatically** every time a consumer purchases energy from the grid. Both Ravi and Priya can see the trade in their **Transaction History** — with a link to the actual blockchain record. **No one can alter or fake this.**

7. The platform shows **Analytics** — how much energy was traded, price trends, grid supply vs demand charts.

 Why Blockchain?

Without Blockchain	With Blockchain
Need a utility company in the middle (takes fees)	Consumer pays producer directly via grid, no middleman
Company can change records	Records are permanent & tamper-proof
Trust the company	Trust the code (smart contract)
Slow settlement (days)	Instant payment when consumer buys from grid
Opaque pricing	Transparent, auditable pricing

 Why ESP32 / IoT?

The ESP32 acts as a **smart meter**. In a real deployment, it would be connected to the solar inverter or meter. For our demo, it sends **simulated but realistic readings** (solar production follows a bell curve — peaks at noon, zero at night). This proves our platform can work with **real hardware**, not just fake data in the browser.

 The Complete Flow (Step by Step)

Step 1:

ESP32 smart meter powers on, connects to WiFi
↳ Starts sending energy readings every 5 seconds to our server

Step 2:

Ravi (Producer) opens the website
↳ Clicks "Connect Wallet" → MetaMask popup → signs in

Step 3:

Ravi registers as the "Producer"
↳ Transaction sent to blockchain → producer recorded permanently

Step 4:

Ravi's dashboard shows live data from his ESP32 meter
↳ He sees: producing 4.2 kW, consuming 1.1 kW, surplus 3.1 kW

Step 5:

Ravi clicks "Feed Grid"
↳ His surplus energy (3.1 kW) is pushed into the grid
↳ Grid supply updated on blockchain (transparent, can't be faked)

Step 6:

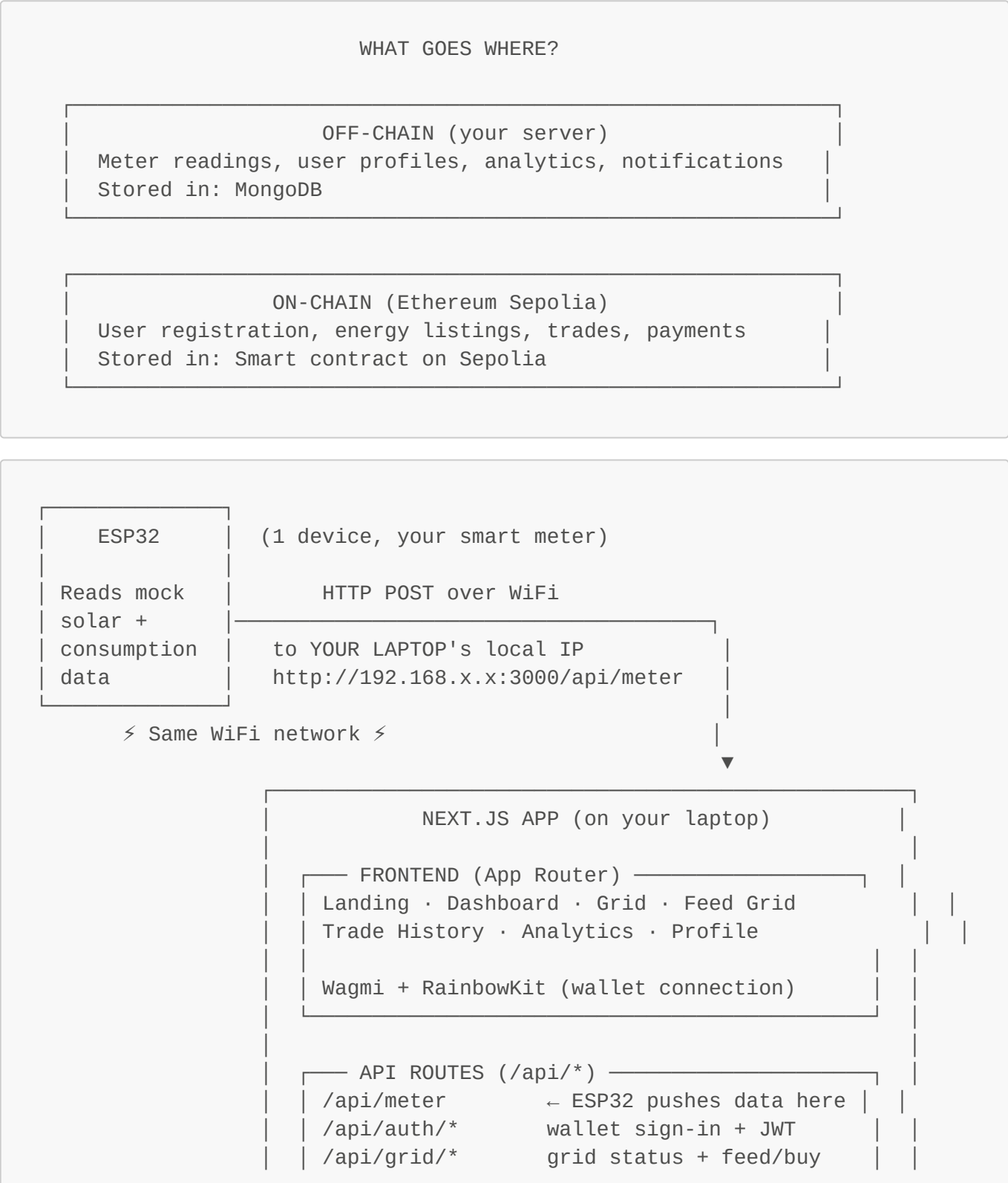
Priya opens the website, connects wallet, registers as "Consumer"
↳ Goes to Grid Dashboard → sees available energy in the grid

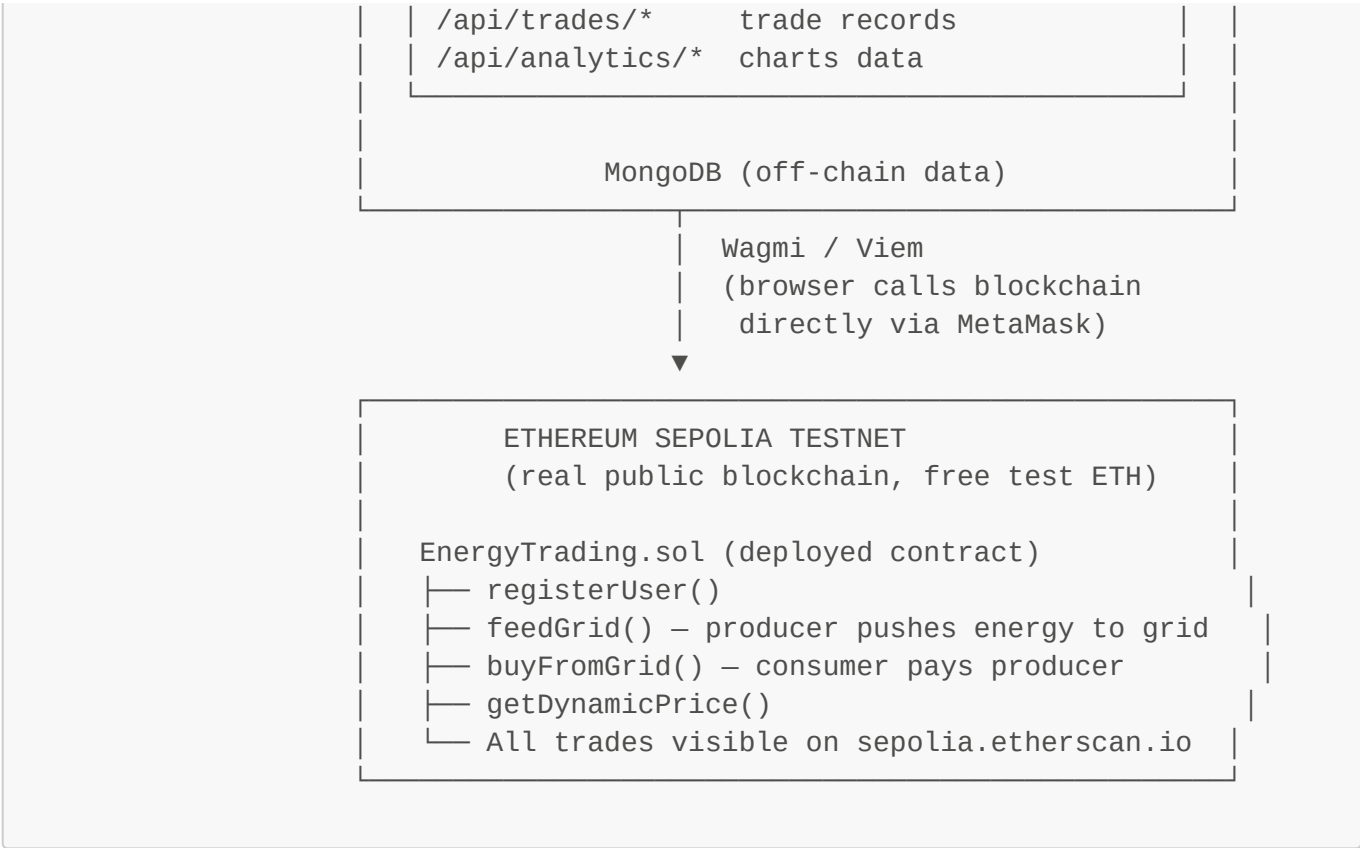
Step 7:

Priya clicks "Buy 20 kWh from Grid"
↳ Smart contract calculates: $20 \times 0.001 = 0.02$ ETH
↳ 0.02 ETH transfers from Priya → Ravi (the producer) automatically
↳ Trade recorded on blockchain forever

- Step 8: Ravi gets paid! Both see the trade in their history
↳ Click the transaction hash → see it on Sepolia Etherscan
- Step 9: Analytics page shows grid-wide stats
↳ Total energy traded, grid supply, consumer demand, price trends

2. Architecture Diagram





3. Tech Stack

Layer	Technology	Why
Fullstack	Next.js 14 (App Router)	Single codebase for frontend + API
Styling	Tailwind CSS	Rapid prototyping
Wallet	RainbowKit + Wagmi	Best-in-class wallet UX
Database	MongoDB (Mongoose)	Flexible schema, fast setup
Blockchain	Solidity + Foundry	Faster compilation, better testing, forge
Network	Ethereum Sepolia Testnet	Real public testnet, free ETH from faucet
IoT	1× ESP32 (Arduino C++)	Real hardware smart meter
Charts	Recharts	React charting library
Auth	MetaMask wallet sign + JWT	Web3-native auth

4. Smart Contract Design

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract EnergyTrading {
```

```

enum Role { Producer, Consumer }

struct User {
    string  name;
    Role    role;
    bool    isRegistered;
}

// The grid – fed by the single producer
struct Grid {
    address producer;          // The one producer's wallet
    uint256 totalSupply;       // Wh available in the grid
    uint256 pricePerUnit;      // wei per Wh (base price)
    uint256 lastUpdated;
}

struct Trade {
    uint256 id;
    address buyer;             // Consumer who bought
    address producer;          // Producer who gets paid
    uint256 energyAmount;      // Wh purchased
    uint256 totalPrice;        // Total wei paid
    uint256 timestamp;
}

Grid    public grid;
mapping(address => User) public users;
Trade[] public trades;

uint256 public totalEnergyTraded;
uint256 public totalTransactions;

// Events
event UserRegistered(address indexed user, string name, Role role);
event GridFed(address indexed producer, uint256 amount, uint256
pricePerUnit);
event EnergyPurchased(uint256 indexed tradeId, address indexed buyer,
address indexed producer, uint256 amount, uint256 price);

function registerUser(string memory _name, Role _role) external;
function feedGrid(uint256 _amount, uint256 _pricePerUnit) external;
// Producer feeds energy into grid
function buyFromGrid(uint256 _amount) external payable;
// Consumer buys from grid, payment goes to producer
function getGridStatus() external view returns (Grid memory);
function getUserTrades(address _user) external view returns (Trade[]
memory);
function getDynamicPrice() public view returns (uint256);
// Price based on grid supply vs demand
}

```

Dynamic Pricing

Price goes UP when lots of consumers want energy (high demand on the grid).
Price goes DOWN when the producer has fed more energy than consumers need (grid surplus).

Formula:

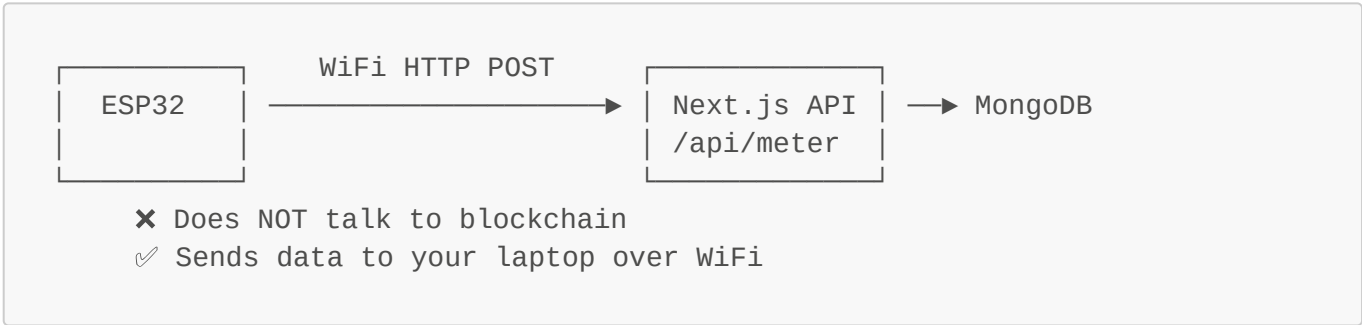
Demand Score = energy bought recently / grid supply available

Score > 1.5 → price × 1.20 (high demand, price goes up 20%)
Score > 1.0 → price × 1.10 (moderate demand, up 10%)
Score < 0.5 → price × 0.85 (grid surplus, discount 15%)
Otherwise → base price set by producer (no change)

5. ESP32 Smart Meter — WHERE and HOW

Where Does the ESP32 Send Data?

To your Next.js server (MongoDB) — NOT to the blockchain.



Why Not Send Directly to Blockchain?

Reason	Explanation
ESP32 can't sign transactions	Too limited (no crypto library, no wallet)
Gas costs	Every 5 seconds × gas fee = way too expensive
Speed	Blockchain is slow (15s blocks), readings need to be instant
Not needed	Meter data is just monitoring — only TRADES need blockchain

How It Works (Step by Step)

1. ESP32 powers on → connects to YOUR WiFi (same network as laptop)
 2. ESP32 finds your laptop at 192.168.x.x (your local IP)
 3. Every 5 seconds, ESP32 sends an HTTP POST request:
URL: http://192.168.1.105:3000/api/meter
Body: {
 "meterId": "METER_001",

```
    "walletAddress": "0xRavi...",
    "production": 3200,      ← Wh (mock solar output)
    "consumption": 1100,    ← Wh (mock home usage)
    "surplus": 2100        ← production - consumption
  }
```

- 4. Next.js /api/meter route receives it:
 - Validates the API key (simple auth)
 - Saves to MongoDB MeterReadings collection
 - Returns { success: true }
- 5. Dashboard page polls /api/meter/[walletAddress] every 3 seconds
 - Fetches latest readings from MongoDB
 - Updates the live chart in the browser

What Data Goes Where?

Data	Stored In	Why
Meter readings (production, consumption)	MongoDB (off-chain)	Fast writes, no gas cost, for dashboards
User registration	Sepolia blockchain	Permanent, tamper-proof identity
Grid supply (producer feeding grid)	Sepolia blockchain	Transparent, verifiable grid state
Trades (consumer buying from grid)	Sepolia blockchain	Trustless payment to producer + proof
Analytics/charts data	MongoDB (off-chain)	Aggregated from readings + trades

Arduino Code Outline

```
// 1. Connect to WiFi (same network as your laptop)
// 2. Every 5 seconds:
//   - Generate mock solar production (sine curve, peaks at noon)
//   - Generate mock home consumption (random 500-2000W)
//   - Calculate surplus = production - consumption
//   - HTTP POST JSON to http://192.168.x.x:3000/api/meter
//   - Print to Serial Monitor (so judges can see live output)
```

What Judges Will See

- The **ESP32 sitting on the table**, powered via USB
- **Serial monitor** on your laptop showing readings being sent
- **Dashboard** in the browser live-updating as new readings arrive from the ESP32
- "This is real hardware sending real data — not a simulation in the browser"

6. Database Schema (MongoDB)

```
// Users
{ walletAddress, name, role, meterId, location, solarCapacity, createdAt }

// MeterReadings (from ESP32)
{ meterId, walletAddress, production, consumption, surplus, timestamp }

// Notifications
{ walletAddress, type, message, read, createdAt }
```

7. Pages

#	Route	What It Does
1	/	Landing page — hero, how-it-works, stats, CTA
2	/dashboard	Live ESP32 data, grid status, producer/consumer stats
3	/grid	View grid supply, dynamic price, buy energy from grid
4	/feed-grid	Producer feeds surplus energy into the grid
5	/history	All trades with blockchain links (payments to producer)
6	/analytics	Charts — grid supply vs demand, price trends
7	/profile	Wallet, role, linked ESP32 meter

8. Folder Structure

```
hack/
├── blockchain/                                # Foundry project
│   ├── src/
│   │   └── EnergyTrading.sol
│   ├── test/
│   │   └── EnergyTrading.t.sol
│   ├── script/
│   │   └── Deploy.s.sol
│   └── foundry.toml
├── esp32/
│   └── smart_meter/
│       └── smart_meter.ino
├── src/                                        # Next.js app
│   ├── app/
│   │   ├── layout.tsx
│   │   └── page.tsx
```



```
|
|
|   ├── dashboard/page.tsx
|   ├── grid/page.tsx
|   ├── feed-grid/page.tsx
|   ├── history/page.tsx
|   ├── analytics/page.tsx
|   ├── profile/page.tsx
|   └── api/
|       ├── auth/nonce/route.ts
|       ├── auth/verify/route.ts
|       ├── grid/route.ts
|       ├── trades/route.ts
|       ├── meter/route.ts
|       └── analytics/route.ts
|
|   ├── components/
|   ├── hooks/
|   ├── lib/
|   ├── models/
|   └── providers/
|
| ├── public/
| ├── tailwind.config.ts
| ├── next.config.js
| ├── package.json
| ├── .env.local
| └── plan.md
```

9. 52-Hour Timeline

Phase 1: Foundation (Hours 0–10)

Hour	Task
0–1	Next.js init + Tailwind + deps (wagmi, rainbowkit, mongoose)
1–3	<code>EnergyTrading.sol</code> — register, feedGrid, buyFromGrid, dynamic price
3–4	Foundry tests (<code>forge test</code>) + deploy script
4–6	MongoDB models + <code>/api/auth/*</code> routes
6–8	<code>/api/meter</code> (ESP32 ingest) + <code>/api/grid</code> routes
8–10	ESP32 Arduino code + test with live API

Phase 2: Core Features (Hours 10–26)

Hour	Task
10–12	Web3Provider + RainbowKit wallet connect
12–15	Landing page (hero, how-it-works, animated stats)
15–18	Dashboard (live ESP32 data, grid status)

Hour	Task
18–21	Grid page (view supply, dynamic price, buy from grid)
21–23	Feed Grid form → smart contract call
23–26	Trade History page (from contract events, showing payments to producer)

Phase 3: Polish (Hours 26–40)

Hour	Task
26–29	Analytics page (Recharts)
29–31	Profile page
31–34	Dynamic pricing end-to-end
34–36	UI polish: animations, dark mode, responsive
36–40	Bug fixes, error handling, edge cases

Phase 4: Demo (Hours 40–52)

Hour	Task
40–43	End-to-end test: full trade flow with 2 wallets
43–45	Seed demo data
45–48	Demo rehearsal + presentation
48–52	Final polish + buffer

10. Auth Flow

1. Click "Connect Wallet" → MetaMask popup
2. Sign a random message to prove you own the wallet
3. Server verifies signature → gives you a login token
4. First time? → Pick your role (Producer / Consumer)
5. Role stored on blockchain – permanent, transparent
6. Only ONE wallet can be the Producer – all others are Consumers

11. Foundry Commands

```
# Install Foundry
curl -L https://foundry.paradigm.xyz | bash
foundryup

# Compile
```

```

cd blockchain && forge build

# Test (locally)
forge test -vvv

# Local testing (optional — use Anvil)
anvil # start local chain in terminal 1
forge script script/Deploy.s.sol --rpc-url http://127.0.0.1:8545 --broadcast

# Deploy to Sepolia (for real demo)
forge script script/Deploy.s.sol \
  --rpc-url https://eth-sepolia.g.alchemy.com/v2/YOUR_KEY \
  --private-key YOUR_DEPLOYER_PRIVATE_KEY \
  --broadcast \
  --verify

# Get free Sepolia ETH
# → https://sepoliafaucet.com or
https://www.alchemy.com/faucets/ethereum-sepolia

```

12. Demo Script (for Judges)

Setup: Laptop running the website + ESP32 on the table + 2 MetaMask wallets (with Sepolia ETH)

1. **"This ESP32 is our smart meter."** Point to it → show serial monitor with live readings
2. **Open the landing page** — show the mission, how-it-works section
3. **Connect Wallet A** (Ravi the Producer) → register → dashboard shows live ESP32 data
4. **Ravi clicks "Feed Grid"** → surplus energy flows into the grid → show MetaMask popup → tx confirmed on Sepolia
5. **Connect Wallet B** (Priya the Consumer) → register → go to Grid Dashboard
6. **Priya buys 20 kWh from the grid** → smart contract runs → Sepolia ETH transfers from Priya to Ravi
7. **"Ravi got paid!"** → show both trade histories → click tx hash → **opens sepolia.etherscan.io** (real blockchain proof!)
8. **Show analytics** — live charts of grid supply vs consumer demand
9. **Key points to emphasize:**
 - "This is deployed on a **real Ethereum testnet** — Sepolia. Same tech as Ethereum mainnet."
 - "No middleman — the producer feeds the grid, consumers buy from it, and the producer gets paid directly"
 - "Every trade is verifiable on Etherscan — can't be faked"
 - "Real IoT hardware sending real data, not a browser simulation"
 - "Dynamic pricing — fair for everyone, adjusts based on grid supply and demand"

 **Priority:** Smart contract + one complete grid trade flow (producer feeds grid → consumer buys from grid → producer gets paid) **first**. Then ESP32. Then pretty UI. A working 1-trade demo beats a pretty but broken full platform.